

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY POLITEHNICA
BUCHAREST
Faculty of Electronics, Telecommunications and Information Technology

MES Project

*Custom driver implementation for a pure-software
UART protocol via bit-banging*

Author: Miron Andrei-Auraş, ACES master

Contents

1. Project description.....	2
2. Hardware description	3
3. Software description.....	6
4. Problems and solutions (optional).....	22
5. Conclusions	22
6. Source code	25

1. Project description

In the field of embedded systems, any microcontroller comes with a wide range of communication protocols that can be used, these being implemented at the hardware level using digital circuits, which allows very good performance and also enables easy configuration and direct integration into applications. However, there may be microcontrollers that do not natively include communication protocols, or complex computing systems that also do not have synchronous or asynchronous protocols implemented at the electrical hardware level (such as SPI, I2C, UART, USART, etc.), and in this case a technique called bit-banging comes into play. This technique allows the implementation and construction of a protocol entirely in software, simulating its functionality using timers, interrupts, and input/output ports. Therefore, even if a system does not have native hardware support for such a protocol, a software version using bit-banging can be used, which is very inexpensive, since no additional hardware module is required to support the emulated protocol, but it will clearly introduce some latency into the system because the CPU will be occupied with processing the protocol.

The project consists of developing the functionality of the asynchronous UART protocol from a purely software point of view, using only common resources available on any microcontroller, such as periodic timers, interrupts, GPIO configured as inputs with interrupts, and GPIO configured as outputs. This means that both data transmission and data reception must be implemented while respecting the fundamental parameters of the UART protocol. At the end, the goal is to be able to transmit and receive messages both to and from a computer and another microcontroller, using the lowest possible baud rate for testing, so that the protocol can be integrated even on very low-performance microcontrollers.

The UART protocol is one of the standards of asynchronous communication in the field of electronics. It transmits and receives bits directly as periodic signals of 0 and 1, represented as voltage levels on the data bus. The UART signal has the following main structure: a start bit, a payload which generally contains 8 bits, an optional parity bit, and a stop bit. From this description, it can be deduced that the signal has multiple states and that the receiver must determine the moment when reception starts, this being the role of the start bit. In the IDLE state, the signal has a HIGH value of 1, while the start bit has a value of 0. Practically, the receiver must detect the first falling edge that appears, which marks the moment when reception begins. Data is transmitted using a specific baud rate, meaning a certain number of bits per second, therefore each bit has the same transmission period, creating a periodic sampling and emission. For values equal to 1, a logical 1 is transmitted for the entire period T_s , and vice versa for values equal to 0. The parity bit is used for error detection, but not for error correction. Using the parity bit, depending on whether even or odd parity is selected, the number of bits equal to 1 from the payload plus the parity bit can be counted, and this number must be even or odd, depending on how the protocol was initially configured (Figure1).

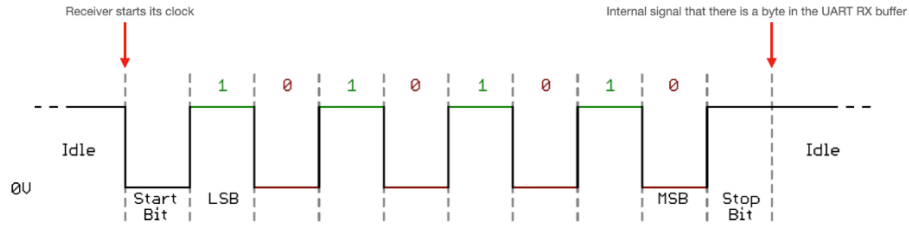


Figure1 UART protocol waveform

2. Hardware description

Initially, before presenting the hardware, I had to plan how the protocol would work from a purely software point of view, to know what resources to use. The system design revolves around the microcontroller timer. Looking at the protocol figure, we can see that we could use a timer with a period equal to the baud rate at which we want to generate the signal. However, for reception, this becomes very difficult. For efficient data communication, timing for sampling is essential, so I decided to do sampling at the middle of each period of the set baud rate. This means that I need to use a timer with an interrupt frequency double that of the baud rate of the signal I want to recreate.

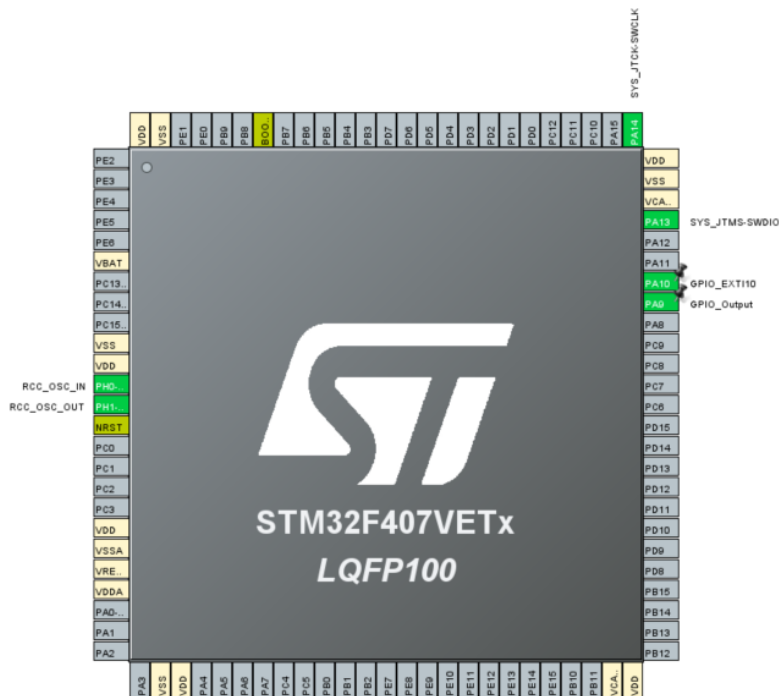


Figure2 Hardware settings

For the project, an ARM STM32F407VET6 MCU from STMicroelectronics was used, on an unofficial board. Using the STM32CubeMX software with STM32CubeIDE, I configured the microcontroller resources efficiently through the graphical interface. On the board, there is also an external quartz clock, which helped me configure the system clock frequency to the maximum of 168

MHz. To test the software implementation of the UART protocol with my laptop, it was necessary to use a USB-TTL converter so that data could be transmitted between these devices.

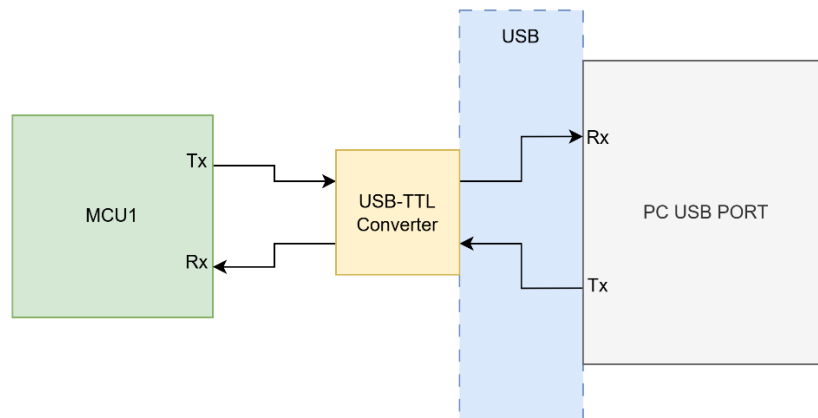


Figure3 Block diagram of the system

Thinking about the protocol and its way of functioning, we conclude that certain resources are needed to make it functional: 2 GPIOs, a transceiver and receiver with falling edge interrupt, and a timer that generates periodic interrupts. I configured the microcontroller accordingly. I used pin PA9 as a simple output, and pin PA10 as an input configured in GPIO external interrupt mode with falling edge trigger detection, to receive the start bit and begin acquisition. It was configured with a pull-up to keep the IDLE signal stable at HIGH.

Pin Name	Signal on Pin	GPIO output le.	GPIO mode	GPt.	Ma.	Use.	Mo.
PA9	n/a	Low	Output Push Pull	No ...	Low		☐
PA10	n/a	n/a	External Interrupt Mode with Falling edge trigger detection	Pull...	n/a		☒

Figure4 GPIO configuration

To configure the timer, besides using the full system clock and activating the external 8 MHz quartz clock, I had to choose a suitable baud rate. Since the lowest available baud rate supported by the device is 300 Hz, I decided to use a timer that generates interrupts at a double frequency of 600 Hz, to respect the design method of the software algorithm anticipated earlier, having the GPIOs configured correctly (the timer source will be the internal clock).

Counter Settings	
Prescaler (PSC - 16 bits value)	3
Counter Mode	Up
Counter Period (AutoReload Register - 32 bits value)	34999
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable
Trigger Output (TRGO) Parameters	
Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
Trigger Event Selection	Reset (UG bit from TIMx_EGR)

Figure5 Timer Configuration

For the timer, I enabled the TIM2 global interrupt in NVIC, and considering the initial input frequency on APB1 of 84 MHz, I divided it by the prescaler of 4, obtaining 21 MHz, and then further divided by 35,000, obtaining exactly 600 Hz. In the end, two essential interrupts are obtained: the periodic interrupt every 1/600 seconds and the interrupt triggered by the falling edge of the GPIO receiver.

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
Non maskable interrupt	☒	0	0
Hard fault interrupt	☒	0	0
Memory management fault	☒	0	0
Pre-fetch fault, memory access fault	☒	0	0
Undefined instruction or illegal state	☒	0	0
System service call via SWI instruction	☒	0	0
Debug monitor	☒	0	0
Pendable request for system service	☒	0	0
Time base: System tick timer	☒	15	0
PVD interrupt through EXTI line 16	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
TIM2 global interrupt	☒	0	0
EXTI line[15:10] interrupts	☒	0	0
FPU global interrupt	☐	0	0

Figure6 Interrupt table NVIC

3. Software description

The software implementation of the protocol was developed in the C programming language using the STM32CubeIDE environment. As mentioned in the previous chapters, it is based on emulating the signal waveform by using GPIOs, interrupt routines, timers, and a robust logic for signal acquisition and detection of the start of a new signal. From a system design perspective, a finite state machine (FSM) was used in order to modularize the signal states and to split the overall algorithm according to the divide and conquer principle, simplifying both development and debugging. Therefore, several subchapters will gradually describe the software development process, starting with the representation of the defining structure of the UART data packet, the function used for parity check, the interrupt service routines (ISRs), static inline functions used to optimize processing time, and finally the implementation of the FSM along with the flags that are used.

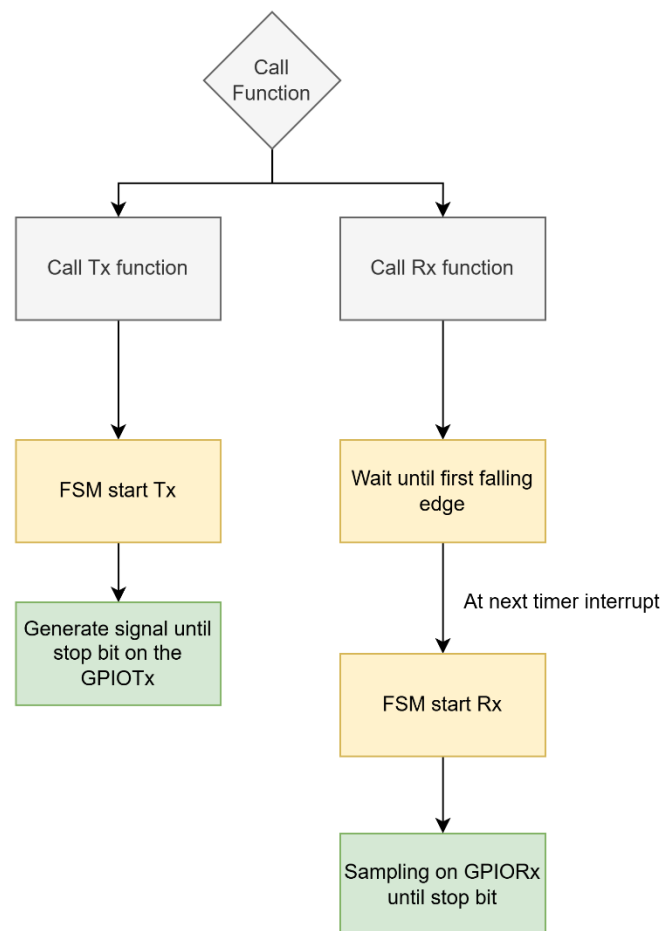


Figure7 Software Workflow Diagram

The project is structured around two working files: a source file named `uart.c` and a header file named `uart.h`. The header file defines the main functions that are available to the user for initializing the protocol parameters, transmitting data, and receiving data. The source file contains the actual implementation of these main functions, as well as a large number of additional static functions and interrupt service routines (ISRs), together with data structures and ENUMs used throughout the project development.

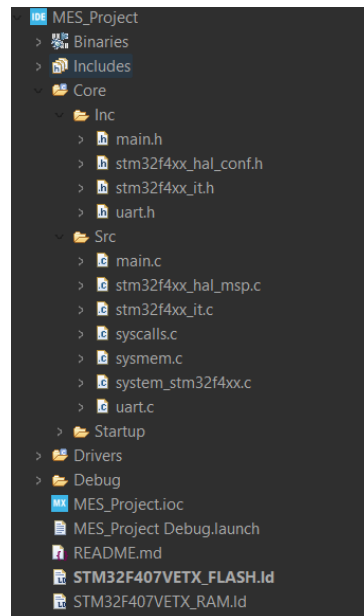


Figure8 Project Files

3.1 Software structure for the UART protocol

From a software perspective, it was necessary to define how the UART configuration data and the actual protocol data would be represented. In order to keep the data compact and easily accessible with minimal processing overhead, a data structure was chosen. This structure contains three main fields: the data field, defined as a `uint8_t`, used to store the transmitted and received data, considering that the UART protocol is defined on 8 bits; the state field, defined by an ENUM called `STATE`, which includes all system states such as `IDLE`, `TX`, `EndTX`, `RX`, `EndRX`, and `ERR`, states based on which the FSM operates inside the periodic interrupt (detailed explanations are provided in the FSM system design subchapter); and the `PARITY` field, also defined using an ENUM, which includes three parity options: `None`, `Odd`, and `Even`, allowing the user to select the desired parity mode during configuration by modifying the associated field. Within the source file, a static variable of the UART data type, named `bufferUart`, was defined and is used to transfer data between the user-provided input and the reception and transmission processes handled by the FSM.


```

8  #include "uart.h"
9
10 extern TIM_HandleTypeDef htim2;
11
12 typedef enum STATE
13 {
14     IDLE,
15     TX,
16     EndTX,
17     RX,
18     EndRX,
19     ERR
20 }STATE;
21
22
23
24 typedef enum PARITY
25 {
26     None,
27     Odd,
28     Even
29 }PARITY;
30
31
32
33 typedef struct UART
34 {
35     uint8_t data;
36
37     STATE state;
38     PARITY parity;
39 }UART;
40
41
42
43 static UART bufferUart = {0,0,0}; /*init uart buffer*/

```

Figure9 UART Software Structure

3.2 Parity detection

Before presenting the actual operating mechanism, I consider it important to describe separately the static inline function `uint8_t uart_check_parity(uint8_t data)`. This function returns TRUE if the number of bits set to 1 is even, and 0 if it is odd. The function processes the bits in order to compute the parity in O(1) time and space complexity, thus avoiding the use of loops. I used a custom algorithm that relies only on bit-level operations, such as shifts and logical operations. The core idea of the algorithm is based on an important observation: if we take a pair of bits such as 11, and subtract the left bit from the binary number 11, we always obtain exactly the number of bits set to 1 in that pair. For example, $11 - 1$ in binary results in 10, which is equal to 2, representing the number of bits set to 1 in the pair 11. This observation is also valid for the other three possible pairs: 00, 01, and 10.

```

170 static inline uint8_t uart_check_parity(uint8_t data)
171 {
172     /*
173      * Function that will return the parity of the
174      * data frame response. It will return true for
175      * even parity and false for odd parity in O(1) time
176      * Input: 8 bit data value
177      * Output: uint8_t 0 or 1
178      */
179
180     data = data - ((data>>1)&0x55);
181     data = (data&0x33) + (data>>2&0x33);
182     data = (data + (data>>4)) & 0x0F;
183
184     if(data % 2 == 0)
185     {
186         return true; /*1*/
187     }
188
189     else
190     {
191         return false; /*0*/
192     }
193 }
194
195

```

Figure10 Parity Check Function

As an example, if the input value is 1010 0011, the number is first shifted to the right, resulting in 0101 0001, and a mask of 0x55 (0101 0101) is applied to extract only the bits located at even positions by performing a logical AND operation, effectively clearing the bits at odd positions. By subtracting this newly obtained value from the original input, the result represents the exact sum of the bits set to 1 for each pair of two bits, with these sums also represented on two bits from right to left. Next, these 2-bit pairs are added together step by step, and finally the two resulting sums are added, obtaining the total number of bits equal to 1 in binary form, without using any loops.

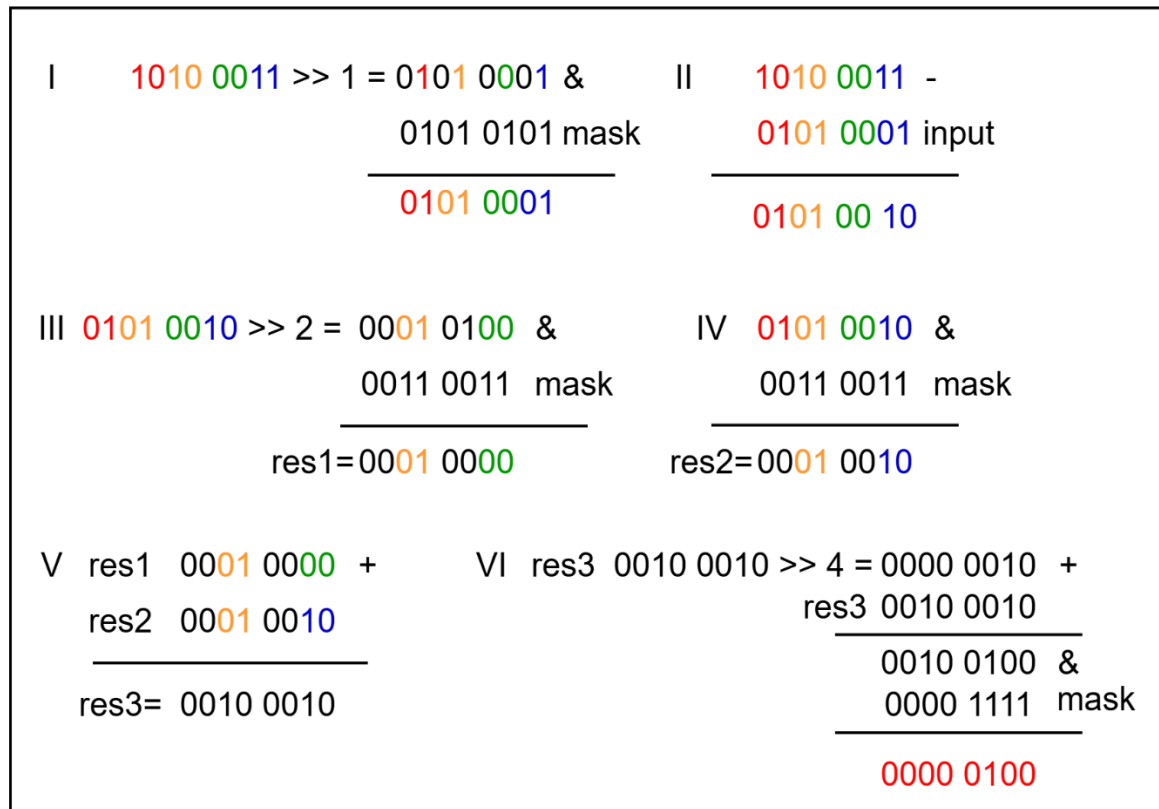


Figure11 Parity Check Algorithm

3.3 Finite State Machine Design

3.3.1 Interrupt Configuration

The FSM operates based on two distinct interrupt sources: a GPIO external interrupt configured on the falling edge for the RX pin, and a periodic timer interrupt (TIM2) used to generate the UART timing base and manage the bit-level protocol logic. The GPIO interrupt is triggered when a falling edge is detected on the RX line, signaling the start bit of an incoming transmission. The timer interrupt, configured to run at twice the baud rate, provides the sampling clock and drives the state transitions within the FSM.

3.3.2 Timing and Sampling Logic

The timing mechanism relies on a static volatile variable *timeUart*, which toggles between 0 and 1 on every timer interrupt call through the modulo operation $timeUart = (timeUart + 1) \% 2$. The volatile qualifier is essential because this variable is modified within an interrupt service routine and read in the main execution context, preventing compiler optimizations that might cache its value incorrectly. This creates a two-phase sampling system where $timeUart = 0$ represents the beginning of a bit period and $timeUart = 1$ represents the midpoint of the bit period. This dual-phase approach is critical for proper UART operation: during transmission, bit transitions occur when $timeUart = 0$, ensuring stable signal levels when the receiver samples at $timeUart = 1$. During reception, the system samples the incoming data at $timeUart = 1$, which corresponds to the middle of the bit period, maximizing noise immunity and timing margin by avoiding sampling near the transition edges where signal integrity might be compromised. The timer counter register (*TIM2->CNT*) is reset to 0 at the start of each transmission or reception sequence to ensure proper synchronization with the incoming or outgoing bit stream, effectively phase-locking the internal timing to the external signal.

3.3.3 State Transitions and FSM Operation

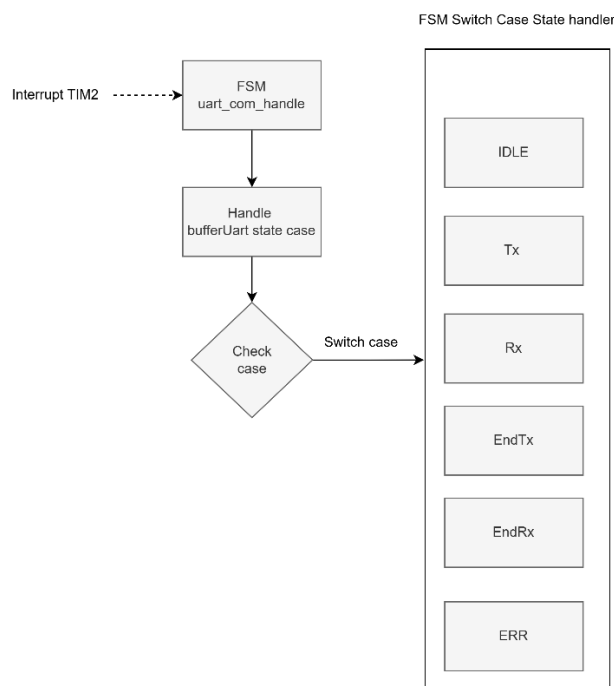


Figure12 FSM Handler Diagram

The system begins in the IDLE state, where no communication activity is taking place and the TX line is held HIGH (mark state) as per UART specification, which defines the idle condition as a logical 1. When the user calls `uart_send_byte()`, the state transitions to TX, the timer counter is reset via `TIM2->CNT = 0` to establish a clean timing reference, `timeUart` is reset to 0 to ensure the first interrupt occurs at the start of a bit period, and the TX pin is immediately pulled LOW to generate the start bit, which signals the beginning of a frame to the receiver. When a falling edge is detected on the RX pin while in IDLE state, the GPIO interrupt fires, triggering the `HAL_GPIO_EXTI_Callback()` function. This callback immediately disables further GPIO interrupts using `HAL_NVIC_DisableIRQ(EXTI15_10_IRQn)` to prevent spurious triggers during the ongoing reception caused by noise or bit transitions within the data frame. The function then resets `flagParityBit` to 0, which is crucial because this flag will be used later to track whether the parity bit has been sampled during reception, and starting with a clean state ensures that previous reception cycles do not interfere with the current one. The `flagRxStart` flag is set to 1 to indicate that the reception has just begun and that the system should wait through the start bit before beginning data sampling. The `timeUart` variable is reset to 0 to synchronize the internal timing with the falling edge of the start bit, ensuring that the first sample occurs at the correct phase. The `bitIndex` variable is reset to 0 because we are about to receive a new byte starting from bit 0. The `bufferUart.data` field is initialized to 0 to clear any residual data from previous receptions, providing a clean slate for accumulating the incoming bits. The timer counter `TIM2->CNT` is reset to 0 to align the periodic interrupts with the start of the received frame. Finally, the state is set to RX, which signals the FSM to begin processing the incoming data on subsequent timer interrupts. The GPIO interrupt remains disabled until the reception completes, at which point it is re-enabled by the `reset_rx_intr()` function to allow the system to detect the next incoming frame.

3.3.4 Transmission State Machine Logic

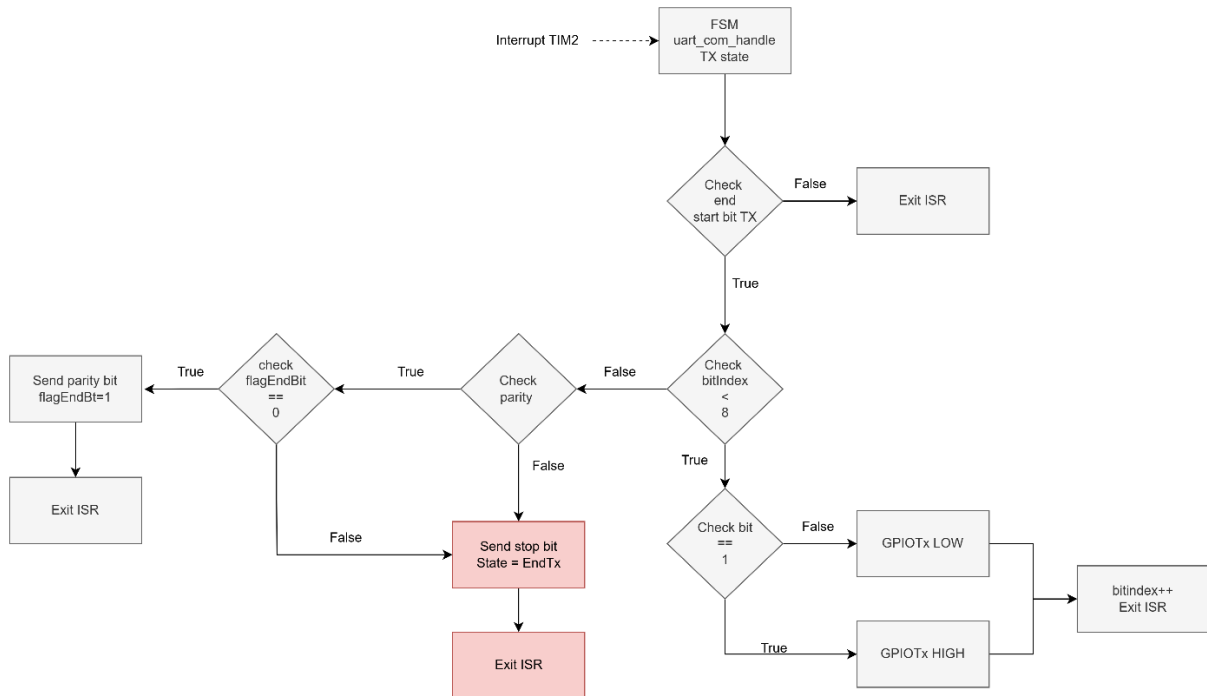


Figure13 Transmission State Machine Diagram

The transmission process is handled entirely within the `uart_com_handle()` function, which is called periodically by the timer interrupt at twice the baud rate. Several static volatile flags control the transmission flow, all declared as volatile because they are modified and read across interrupt and main execution contexts. The `flagStartBit` indicates whether the start bit transmission has completed; it begins at 0 when `uart_send_byte()` is called and transitions to 1 after the first timer interrupt in TX state, marking the end of the start bit period. The `flagDataTx` enables the data payload transmission phase; when set to 1, it signals that the system should begin transmitting the 8 data bits sequentially. The `flagSendBit` tracks the current state of individual bit transmission within helper functions like `uart_send_bit()`, operating as a mini-state machine within a single bit period to ensure the bit value is written only once at `timeUart = 0` and held stable until the next `timeUart = 0`. The `flagEndBit` signals when a bit transmission cycle has finished within the helper function, indicating that the system can proceed to the next bit or phase. The `flagParityBit` stores the computed parity value for the data frame, calculated by `uart_check_parity()` and used to determine what parity bit value should be transmitted. Additionally, the `bitIndex` variable serves as a counter tracking which bit position (0 through 7) is currently being transmitted or received. When in the TX state, the function first checks the condition `if(timeUart == 0 && flagStartBit == 0)`. This check occurs during the first `timeUart = 0` phase after entering the TX state, which corresponds to the end of the start bit period (the start bit was initiated by pulling the TX pin LOW in `uart_send_byte()`, and has now lasted for one complete bit period). At this point, `flagStartBit` is set to 1 to indicate that the start bit transmission has completed and should not be processed again. Simultaneously, `flagDataTx` is set to 1 to enable the data payload transmission phase, signaling that subsequent iterations should focus on transmitting the actual data bits. The `bitIndex` is initialized to 0 because transmission of the 8 data bits will begin from the least significant bit (LSB-first transmission as per standard UART protocol). Once `flagDataTx = 1`, the system enters the data payload transmission phase. The condition `if(bitIndex < 8 && timeUart == 0)` ensures that data bits are transmitted only at the beginning of each bit period and only for the 8 data bits (indices 0 through 7). For each bit position, the corresponding bit is extracted from `bufferUart.data` using a bit mask (`bufferUart.data & (1 << bitIndex)`). The expression `(1 << bitIndex)` creates a mask with only the bit at position `bitIndex` set to 1; for example, when `bitIndex = 2`, the mask is `0000 0100`. The AND operation isolates that specific bit from the data byte. The result is then compared with 0 using `!= 0` to determine if the bit is set or clear. If the bit is set (non-zero result), the TX pin is driven HIGH using `HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_SET)`, representing a logical 1. If the bit is clear (zero result), the TX pin is driven LOW using `HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_RESET)`, representing a logical 0. After writing the appropriate bit value, `bitIndex` is incremented by 1 to prepare for the next bit on the subsequent `timeUart = 0` phase. The function then immediately returns, which is critical because it prevents any further state changes within this interrupt call. This return ensures that the current bit value remains stable on the TX line for the entire bit period. The bit will continue to hold its value through the `timeUart = 1` phase (when the receiver samples it) and only change when `timeUart` cycles back to 0 on the next timer interrupt, at which point `bitIndex` will have been incremented and the next bit will be transmitted. When `bitIndex` reaches 8 and `timeUart = 0`, all 8 data bits have been transmitted, and the system moves to the parity bit transmission phase. The condition `if(bitIndex == 8 && timeUart == 0)` captures this transition point. First, the function checks `if(bufferUart.parity == None)`, which determines whether parity is disabled in the configuration. If parity is disabled, there is no parity bit to transmit, so the function immediately calls `uart_send_stop_bit()` to skip the parity phase and proceed directly to transmitting the stop bit. The function then returns to prevent further execution in this iteration. If parity is enabled (`bufferUart.parity` is either Odd or Even), the system

proceeds to transmit the parity bit. The *flagEndBit* flag is checked to determine the current sub-phase of parity bit transmission. When *flagEndBit* = 0, it indicates that the parity bit has not yet been transmitted, so the system must initiate the transmission. The parity bit value is determined based on the configured parity mode and the value stored in *flagParityBit*, which was computed earlier by calling *uart_check_parity(bufferUart.data)* in the *uart_send_byte()* function. Recall that *uart_check_parity()* returns TRUE (1) if the number of 1-bits in the data is even, and FALSE (0) if it is odd. For Even parity mode (*bufferUart.parity* == *Even*), the goal is to make the total number of 1-bits (data bits plus parity bit) even. If *flagParityBit* == *true*, this means the data already contains an even number of 1-bits, so the parity bit must be 0 to maintain an even total. Therefore, *uart_send_bit(0)* is called. Conversely, if *flagParityBit* == *false*, the data contains an odd number of 1-bits, so the parity bit must be 1 to achieve an even total, and *uart_send_bit(1)* is called. For Odd parity mode (*bufferUart.parity* == *Odd*), the logic is inverted: the goal is to make the total number of 1-bits odd. If *flagParityBit* == *true* (even number of 1-bits in data), the parity bit must be 1, so *uart_send_bit(1)* is called. If *flagParityBit* == *false* (odd number of 1-bits in data), the parity bit must be 0, so *uart_send_bit(0)* is called. The *uart_send_bit()* helper function operates as a three-phase mini-state machine. When called with a bit value, it first checks *if(timeUart == 0 && flagSendBit == 0)*. This condition is true during the initial call at the start of a bit period. At this point, *flagSendBit* is set to 1 to indicate that the bit transmission has been initiated, and the TX pin is set to the specified bit value using *HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, bit)*. The function then returns immediately, allowing the bit to hold for the remainder of the bit period. On the next timer interrupt, *timeUart* will be 1, and the condition *if(timeUart == 1 && flagSendBit == 1)* will be true. This phase represents the middle of the bit period, and the function simply returns without action, allowing the bit to remain stable while the receiver samples it. On the subsequent timer interrupt, *timeUart* returns to 0, and the condition *if(timeUart == 0 && flagSendBit == 1)* becomes true. This signals the end of the bit period. At this point, *flagSendBit* is reset to 0 to prepare for the next bit transmission, and *flagEndBit* is set to 1 to signal to the calling code that the bit transmission has completed. Returning to the parity transmission logic in *uart_com_handle()*, after *uart_send_bit()* completes its three-phase cycle, *flagEndBit* will be 1 on the next iteration when *bitIndex* == 8 && *timeUart* == 0. The condition *if(flagEndBit == 1)* then becomes true, indicating that the parity bit transmission has finished. At this point, the function calls *uart_send_stop_bit()* to initiate the stop bit transmission, which will occur on the next *timeUart* = 0 phase. The *uart_send_stop_bit()* function operates similarly to *uart_send_bit()*, using the *flagStopBit* flag to track its state. When first called with *timeUart* = 0 and *flagStopBit* = 0, the TX pin is set HIGH using *HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_SET)* to generate the stop bit (logical 1), *flagStopBit* is set to 1 to indicate that the stop bit transmission has begun, and the function returns. On the next timer interrupt when *timeUart* = 1 and *flagStopBit* = 1, the function simply returns, allowing the stop bit to hold for the full bit period. When *timeUart* cycles back to 0 with *flagStopBit* = 1, the stop bit period has completed. At this point, *flagStopBit* is reset to 0 to prepare for the next transmission, and the state is transitioned to *EndTX*, signaling that the entire frame transmission (start bit, data bits, optional parity bit, and stop bit) has finished. On the next timer interrupt, when the state is *EndTX*, the FSM simply transitions back to *IDLE*, making the system ready to transmit the next byte. The TX line remains HIGH (set during the stop bit transmission), which is the correct idle state for UART. In the *uart_send_data()* function, which handles multi-byte transmissions, a while loop checks *while(bufferUart.state == TX || bufferUart.state == EndTX)* before calling *uart_send_byte()* for the next byte. This polling ensures that the previous byte has fully completed transmission and the system has returned to *IDLE* before initiating the next byte, preventing overlap and maintaining proper frame spacing.

3.3.5 Reception State Machine Logic

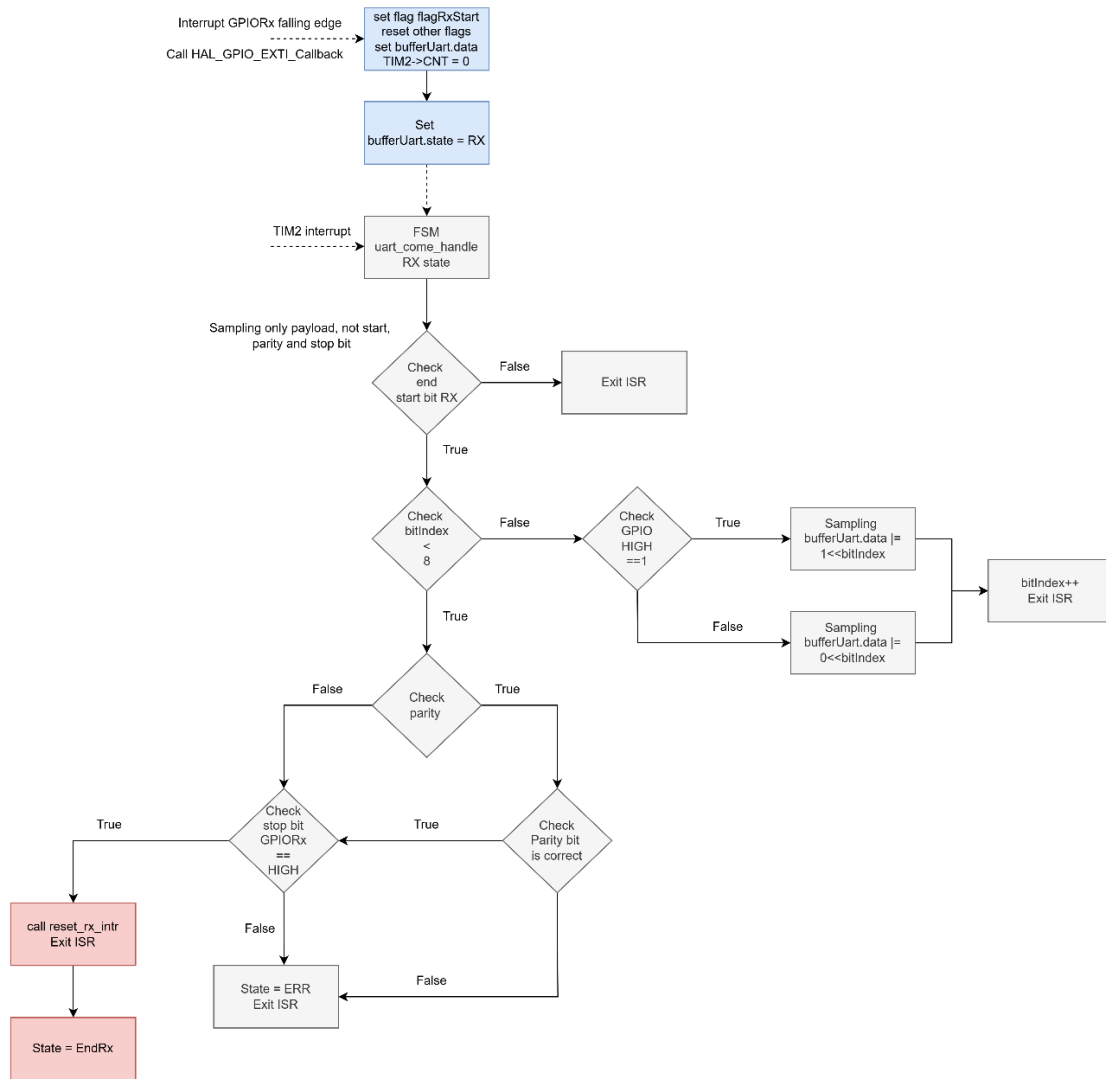


Figure14 Reception State Machine Diagram

The reception process operates within `uart_com_handle()` when the FSM is in the RX state, triggered initially by the GPIO interrupt detecting a falling edge on the RX line. When the `HAL_GPIO_EXTI_Callback()` function is invoked, it sets `flagRxStart` to 1 to indicate that a new reception has just begun, and initializes `bitIndex` to 0 in preparation for receiving the 8 data bits starting from the LSB. During the first timer interrupt after the GPIO event, the condition `if(flagRxStart == 1 && timeUart == 1)` will be true. This represents the midpoint of the start bit period. At this point, the function simply returns without taking any action, effectively waiting through the start bit. This wait is necessary because the start bit is only a synchronization marker and does not carry data. By waiting until `timeUart` cycles back to 0, the system ensures that the subsequent data bit sampling will be properly aligned. On the next interrupt when `timeUart` returns to 0, the condition `if(flagRxStart == 1 && timeUart == 1)` is false, so execution continues to the else block where `flagRxStart` is cleared to 0. This signals that the start bit period has been processed and normal data

reception can proceed. For each bit position from 0 to 7, the condition *if(bitIndex < 8 && timeUart == 1)* ensures that data bits are sampled only at the midpoint of each bit period (*timeUart = 1*), which is the optimal sampling point for maximizing noise immunity. At this point, the RX pin is read using *HAL_GPIO_ReadPin(UART_RX_PORT, UART_RX_PIN)*, which returns either 1 (GPIO_PIN_SET) if the pin is HIGH or 0 (GPIO_PIN_RESET) if the pin is LOW. If the pin reads HIGH, indicating a logical 1 for this bit position, the corresponding bit in *bufferUart.data* must be set. This is accomplished using the OR operation *bufferUart.data = bufferUart.data | (1 << bitIndex)*. The expression *(1 << bitIndex)* creates a mask with only the bit at position *bitIndex* set to 1, and the OR operation sets that bit in *bufferUart.data* while leaving all other bits unchanged. If the pin reads LOW, indicating a logical 0, the corresponding bit must be cleared using the AND operation with a negated mask: *bufferUart.data = bufferUart.data & ~(1 << bitIndex)*. The expression *~(1 << bitIndex)* creates a mask with all bits set to 1 except the bit at position *bitIndex*, which is 0. The AND operation clears that specific bit while preserving all other bits. After updating *bufferUart.data* with the sampled bit value, *bitIndex* is incremented by 1 to prepare for sampling the next bit, and the function returns. This return is crucial because it ensures that each bit is sampled only once per bit period, at the optimal *timeUart = 1* phase. The function will be called again on the next timer interrupt, but at that point *timeUart* will be 0, so the sampling condition will be false and the bit will not be re-sampled. Only when *timeUart* returns to 1 on the subsequent interrupt will the next bit be sampled. When *bitIndex* reaches 8 and *timeUart = 1*, all 8 data bits have been received and stored in *bufferUart.data*. The system now enters the frame validation phase, which involves checking the optional parity bit (if enabled) and the mandatory stop bit. The condition *if(bitIndex == 8 && timeUart == 1)* captures this transition point. First, the function checks *if(bufferUart.parity == None)* to determine whether parity checking is disabled. If parity is disabled, the system expects only a stop bit immediately after the data bits. The RX pin is sampled using *HAL_GPIO_ReadPin(UART_RX_PORT, UART_RX_PIN)*. If the pin reads HIGH (logical 1), this represents a valid stop bit, indicating a correctly formatted frame. In this case, *reset_rx_intr()* is called to finalize the reception. If the pin reads LOW (logical 0), this represents an invalid stop bit, indicating a framing error. The state is immediately transitioned to ERR using *bufferUart.state = ERR*, and the function returns to prevent further processing of the corrupted frame. The *reset_rx_intr()* function performs several critical cleanup and signaling operations. First, it clears the GPIO interrupt pending flag using *__HAL_GPIO_EXTI_CLEAR_IT(UART_RX_PIN)*. This is essential because a pending interrupt flag from the initial falling edge detection might still be set, and failing to clear it could cause an immediate spurious interrupt when the GPIO interrupt is re-enabled. Next, *flagSequence* is set to 1, which serves as a signal to the user-space code (specifically the *uart_receive_data()* function) that a complete byte has been received and is ready to be read from *bufferUart.data*. The *uart_receive_data()* function contains a while loop *while(flagSequence == 0)* that blocks until this flag is set, effectively implementing a synchronization mechanism between the interrupt-driven reception and the user's blocking API. After setting *flagSequence*, the function re-enables the GPIO interrupt using *HAL_NVIC_EnableIRQ(EXTI15_10_IRQn)*, which allows the system to detect the next incoming frame. Finally, the state is transitioned to EndRX, which will cause the FSM to return to IDLE on the next timer interrupt. If parity is enabled (*bufferUart.parity* is either Odd or Even), the reception logic includes an additional parity bit sampling and validation phase before checking the stop bit. The *parityBitValue* variable is introduced here as a static volatile *uint8_t* to store the received parity bit value. The *flagParityBit* flag, which was initialized to 0 at the start of reception, is used to track whether the parity bit has been sampled yet. When *flagParityBit = 0*, the system is in the parity bit sampling phase. The RX pin is sampled to obtain the parity bit value, which is stored in *parityBitValue* using *parityBitValue = HAL_GPIO_ReadPin(UART_RX_PORT, UART_RX_PIN)*. This variable will hold either 1 or 0 depending on the state of the RX line at this moment. The received parity is then validated by computing *(uart_check_parity(bufferUart.data) +*

$parityBitValue) \% 2$. Recall that `uart_check_parity(bufferUart.data)` returns 1 if the number of 1-bits in the received data is even, and 0 if it is odd. By adding `parityBitValue` (the received parity bit) and taking modulo 2, we obtain a value that should match the expected parity mode. For Even parity mode (`bufferUart.parity == Even`), the sum $(uart_check_parity(bufferUart.data) + parityBitValue) \% 2$ must equal 1 for the parity to be correct. This is because: if the data has an even number of 1-bits (`uart_check_parity` returns 1), the parity bit should be 0, giving $(1 + 0) \% 2 = 1$. If the data has an odd number of 1-bits (`uart_check_parity` returns 0), the parity bit should be 1, giving $(0 + 1) \% 2 = 1$. In both cases, correct even parity results in a sum of 1. If this condition is met, the parity check has passed, so `flagParityBit` is set to 1 to indicate that the parity bit has been successfully sampled and validated, and the function returns to wait for the stop bit on the next `timeUart = 1` phase. For Odd parity mode (`bufferUart.parity == Odd`), the sum must equal 0 for the parity to be correct. This is because: if the data has an even number of 1-bits (`uart_check_parity` returns 1), the parity bit should be 1, giving $(1 + 1) \% 2 = 0$. If the data has an odd number of 1-bits (`uart_check_parity` returns 0), the parity bit should be 0, giving $(0 + 0) \% 2 = 0$. If this condition is met, `flagParityBit` is set to 1 and the function returns. If neither parity condition is satisfied, it means the received parity bit does not match the expected value, indicating a parity error. In this case, the state is immediately transitioned to ERR using `bufferUart.state = ERR`, `flagParityBit` is set to 1 to prevent re-processing, and the function returns to abort further processing of the corrupted frame. On the next timer interrupt, if the parity check passed, `timeUart` will be 1 again and the condition `if(flagParityBit == 1)` will be true, indicating that the system should now check the stop bit. The RX pin is sampled one final time. If it reads HIGH (logical 1), this represents a valid stop bit, and `reset_rx_intr()` is called to finalize the reception as described previously. If the stop bit is invalid (LOW), the state transitions to ERR and the function returns. In the EndRX state, which is entered after `reset_rx_intr()` is called, the system simply transitions back to IDLE on the next timer interrupt. This completes the reception cycle, and the system is now ready to receive the next frame when the GPIO interrupt detects another falling edge. In the ERR state, which is entered when any validation error occurs (invalid stop bit or parity error), the function performs cleanup operations to allow the system to recover. The GPIO interrupt flag is cleared using `__HAL_GPIO_EXTI_CLEAR_IT(UART_RX_PIN)` to prevent spurious interrupts, the interrupt is re-enabled using `HAL_NVIC_EnableIRQ(EXTI15_10_IRQn)` to allow detection of future frames, and the state is transitioned to IDLE. Notably, `flagSequence` is NOT set to 1 in the error case, which means the `uart_receive_data()` function will continue waiting rather than reading corrupted data. The received byte in `bufferUart.data` is discarded, and the system is ready to attempt reception of the next frame. The `uart_receive_data()` function uses the static volatile variable `timerRx` to implement a timeout mechanism. When the function is called with a timeout value in seconds, `timerRx` is initialized to `timeout * 600`. This calculation is based on the assumption that the timer interrupt fires at a rate of 600 Hz (twice the baud rate of 300 baud), so 600 interrupts correspond to approximately 1 second. In the `HAL_TIM_PeriodElapsedCallback()` function, after calling `uart_com_handle()`, the code checks `if(timerRx != 0)` and decrements `timerRx--`. This decrement occurs on every timer interrupt regardless of the FSM state. The `uart_receive_data()` function contains a while loop `while(len != 0 && timerRx != 0)` that continues as long as there are bytes remaining to receive and the timeout has not expired. Inside this loop, the code waits for `flagSequence` to become 1 using `while(flagSequence == 0)`, which is set by `reset_rx_intr()` when a byte is successfully received. Once `flagSequence` is set, it is immediately cleared back to 0 to prepare for the next byte, the received byte is copied from `bufferUart.data` to the user's buffer, the buffer pointer is incremented, and `len` is decremented. If `timerRx` reaches 0 before all bytes are received, the while loop exits, and the function returns with incomplete data, allowing the user to detect timeout conditions by checking how many bytes were actually received.

3.3.6 Helper Functions

uart_init():

The *uart_init()* function serves as the initialization routine for the custom UART protocol implementation for the user. This function accepts a single parameter *parity*, which is an unsigned 8-bit value representing the desired parity mode: 0 for None, 1 for Odd, and 2 for Even. The function performs two critical initialization steps. First, it sets the *bufferUart.parity* field to the user-specified parity mode, configuring how parity bits will be generated during transmission and validated during reception. Second, it initializes *bufferUart.state* to IDLE, establishing the FSM's initial state and ensuring the system is ready to accept transmission or reception requests. Finally, the function sets the TX pin to HIGH using *HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_SET)*, which establishes the proper idle condition for the UART line. According to UART specification, the line must be in a mark state (logical 1) when no transmission is occurring, and this initialization ensures that any receiving device connected to this TX line will correctly interpret the idle state rather than misinterpreting a LOW level as a start bit.

Uart_send_byte():

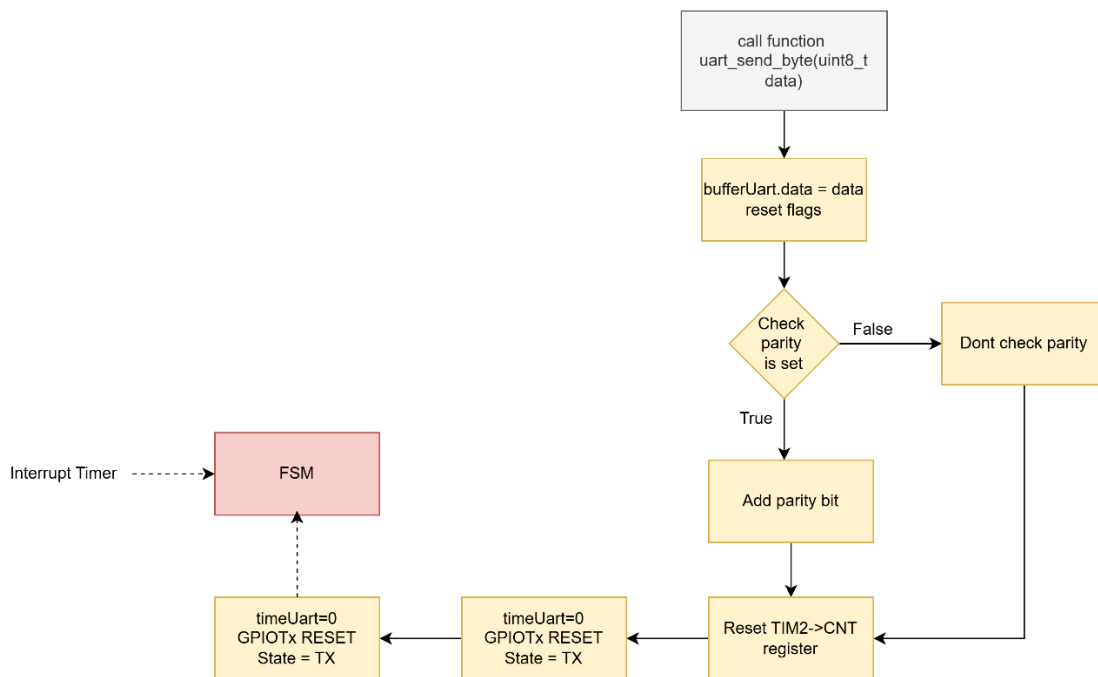


Figure15 Send Byte Diagram

The *uart_send_byte()* function is responsible for initiating the transmission of a single byte through the UART protocol. This function accepts a single parameter *data* of type *uint8_t*, which represents the byte to be transmitted. The function does not actually perform the bit-by-bit transmission itself; instead, it prepares the system state and allows the timer interrupt-driven FSM to handle the actual transmission process. The function begins by storing the data byte in *bufferUart.data*, making it

accessible to the FSM during the transmission process. Next, all transmission-related flags are reset to their initial states: *flagStartBit* is set to 0 to indicate that the start bit transmission has not yet completed, *flagDataTx* is set to 0 to indicate that data transmission has not yet begun, *flagSendBit* is set to 0 to prepare the bit-sending helper function for use, and *flagEndBit* is set to 0 to indicate that no bit transmission cycle has completed yet. This comprehensive flag reset ensures that residual state from previous transmissions does not interfere with the current transmission. If parity is enabled (*bufferUart.parity != None*), the function computes the parity of the data byte by calling *uart_check_parity(bufferUart.data)* and stores the result in *flagParityBit*. This pre-computation allows the FSM to quickly determine the correct parity bit value during the transmission phase without needing to recalculate it. The timer counter is then reset using *TIM2->CNT = 0* to establish a clean timing reference, and *timeUart* is reset to 0 to ensure that the first timer interrupt will occur at the beginning of a bit period. The TX pin is immediately pulled LOW using *HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_RESET)* to generate the start bit, which signals to the receiver that a new frame is beginning. Finally, the state is transitioned to TX, which activates the transmission logic in the *uart_com_handle()* function. From this point forward, the timer interrupt will repeatedly call *uart_com_handle()*, which will manage the step-by-step transmission of the start bit, data bits, optional parity bit, and stop bit.

uart_send_data():

The *uart_send_data()* function provides a higher-level interface for transmitting multiple bytes sequentially through the UART protocol. This function accepts three parameters: a pointer *data* to a character array containing the bytes to be transmitted (marked with the *restrict* keyword to indicate that this pointer is the only means of accessing the underlying data, allowing compiler optimizations), a *len* parameter of type *size_t* indicating the number of bytes to transmit, and operates by repeatedly calling *uart_send_byte()* for each byte in the array. The function uses a while loop *while(len != 0)* that continues until all bytes have been transmitted. Within this loop, an inner while loop *while(bufferUart.state == TX || bufferUart.state == EndTX)* implements a polling mechanism that waits for the FSM to complete the transmission of the previous byte. This polling is necessary because the actual transmission is handled asynchronously by the timer interrupt, and attempting to initiate a new transmission while the previous one is still in progress would cause data corruption or loss. By checking for both TX and EndTX states, the function ensures it waits through the entire transmission cycle, including the final state transition from EndTX to IDLE. Once the FSM returns to IDLE, indicating that the previous byte has been fully transmitted and the TX line has returned to the idle state, the function calls **uart_send_byte(data)* to initiate transmission of the next byte. The data pointer is then incremented using *data++* to point to the next byte in the array, and *len* is decremented using *len--* to track how many bytes remain to be transmitted. This process repeats until *len* reaches 0, at which point all bytes have been queued for transmission and the function returns. It is important to note that this function returns after the last byte has been queued, not necessarily after it has been fully transmitted; the caller must implement their own polling of *bufferUart.state* if they need to ensure the final byte has completed transmission before proceeding.

uart_receive_data():

The *uart_receive_data()* function provides a blocking interface for receiving multiple bytes through the UART protocol with timeout capability. This function accepts three parameters: a pointer *buffer* to a character array where received bytes will be stored, a *len* parameter of type *uint16_t* indicating the number of bytes to receive, and a *timeout* parameter of type *uint16_t* specifying the maximum wait time in seconds. The function begins by initializing the *timerRx* variable using *timerRx = timeout **

600, which converts the timeout from seconds to timer interrupt ticks. The multiplication factor of 600 is derived from the timer interrupt frequency: with a baud rate of 300 and the timer running at twice the baud rate, 600 interrupts occur per second. This timeout mechanism prevents the function from blocking indefinitely if the expected data never arrives or if fewer bytes than requested are received. The function then enters a while loop *while(len != 0 && timerRx != 0)* that continues as long as there are bytes remaining to receive and the timeout has not expired. Inside this loop, an inner while loop *while(flagSequence == 0)* implements a busy-wait that blocks until a complete byte has been received. The *flagSequence* flag is set to 1 by the *reset_rx_intr()* function when the reception FSM successfully receives a valid byte, providing a synchronization mechanism between the interrupt-driven reception process and this user-space function. Once *flagSequence* becomes 1, indicating that a byte is ready, it is immediately cleared back to 0 using *flagSequence = 0* to prepare for the next byte. The received byte is then copied from *bufferUart.data* to the user's buffer using **buffer = bufferUart.data*, the buffer pointer is incremented using *buffer++* to point to the next storage location, and *len* is decremented using *len--* to track how many bytes remain to be received. This process repeats until either all requested bytes have been received (*len* reaches 0) or the timeout expires (*timerRx* reaches 0). The function does not return any value to indicate success or failure, but the caller can determine whether all bytes were received by tracking the original *len* value and comparing it to the final state. If the timeout expires before all bytes are received, the function returns with partial data, and it is the caller's responsibility to detect this condition and handle it appropriately.

uart_send_bit():

The *uart_send_bit()* function is a static inline helper function designed to transmit a single bit value (0 or 1) over the UART TX line while ensuring the bit remains stable for exactly one complete bit period. This function accepts a single parameter *bit* of type *uint8_t*, which should be either 0 or 1, representing the logical value to be transmitted. The function implements a three-phase state machine using the *flagSendBit* flag to track its progress through the bit transmission cycle. In the first phase, when the function is called with *timeUart = 0* and *flagSendBit = 0*, it means we are at the start of a new bit period and the bit has not yet been written to the TX line. At this point, *flagSendBit* is set to 1 to indicate that the bit transmission has been initiated and to prevent the bit from being written multiple times. The TX pin is then set to the specified bit value using *HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, bit)*, where *bit* is passed directly as the GPIO pin state (0 for LOW/RESET, 1 for HIGH/SET). The function immediately returns after setting the pin, allowing the bit value to hold stable on the line. In the second phase, which occurs on the next timer interrupt when *timeUart = 1* and *flagSendBit = 1*, the function is at the midpoint of the bit period. This is the critical sampling point where the receiver will read the bit value, so it is essential that the TX line remains stable. The function simply returns without any action, allowing the bit to continue holding its value. In the third and final phase, which occurs on the subsequent timer interrupt when *timeUart* cycles back to 0 and *flagSendBit = 1*, the function recognizes that a complete bit period has elapsed. At this point, *flagSendBit* is reset to 0 to indicate that the bit transmission is complete and the function is ready to transmit another bit if called again. Simultaneously, *flagEndBit* is set to 1 to signal to the calling code (typically the parity bit transmission logic in *uart_com_handle()*) that the bit transmission cycle has finished and the next phase of transmission can begin. This three-phase approach ensures that each bit remains stable for exactly one bit period, providing the receiver with a reliable sampling window and maintaining strict timing compliance with the UART protocol.

uart_send_stop_bit():

The `uart_send_stop_bit()` function is a static helper function specifically designed to transmit the stop bit, which marks the end of a UART frame. The stop bit is always a logical 1 (HIGH), and like data bits, it must remain stable for exactly one bit period to allow the receiver to properly detect the frame boundary and return to the idle state. This function takes no parameters and operates using a three-phase state machine similar to `uart_send_bit()`, but uses the dedicated `flagStopBit` flag to track its progress. In the first phase, when `timeUart = 0` and `flagStopBit = 0`, the function is being called for the first time to initiate stop bit transmission. At this point, `flagStopBit` is set to 1 to indicate that stop bit transmission has begun, and the TX pin is driven HIGH using

`HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_SET)` to generate the stop bit.

The function then returns immediately, allowing the HIGH state to hold on the TX line. In the second phase, occurring on the next timer interrupt when `timeUart = 1` and `flagStopBit = 1`, the function is at the midpoint of the stop bit period. The function simply returns without any action, ensuring the stop bit remains HIGH for the receiver to sample. The comment in the code states "make sure to end stop bit", which emphasizes that this phase is critical for maintaining the full bit period duration. In the third and final phase, when `timeUart` cycles back to 0 and `flagStopBit = 1`, a complete stop bit period has elapsed. At this point, `flagStopBit` is reset to 0 to prepare for the next transmission, and `bufferUart.state` is transitioned to EndTX to signal that the entire frame transmission is complete. On the subsequent timer interrupt, when the state is EndTX, the FSM in `uart_com_handle()` will transition the state to IDLE, making the system ready for the next transmission. The TX line remains HIGH after the stop bit completes, which automatically establishes the correct idle state for UART, ensuring seamless spacing between consecutive frames.

reset_rx_intr():

The `reset_rx_intr()` function is a static inline helper function that performs critical cleanup and state management operations when a byte has been successfully received. This function takes no parameters and is called exclusively from within the RX state of `uart_com_handle()` after all frame validation checks (stop bit and optional parity bit) have passed. The function performs four essential operations in a specific sequence. First, it clears the GPIO external interrupt pending flag using

`__HAL_GPIO_EXTI_CLEAR_IT(UART_RX_PIN)`. This operation is critical because the falling edge that triggered the initial reception event sets a hardware interrupt pending flag in the EXTI (External Interrupt) peripheral, and this flag remains set even after the interrupt service routine has executed. If this flag is not explicitly cleared before re-enabling the interrupt, the moment

`HAL_NVIC_EnableIRQ()` is called, the NVIC (Nested Vectored Interrupt Controller) will immediately trigger another interrupt, believing there is a new falling edge to process. This would cause spurious interrupt triggers, potentially initiating reception state machines at incorrect times and corrupting data. By clearing the flag first, the function ensures that only genuine new falling edges will trigger future interrupts. Second, the function sets `flagSequence` to 1 using `flagSequence = 1`. This flag serves as a synchronization signal to the user-space `uart_receive_data()` function, indicating that a complete, valid byte has been received and is available in `bufferUart.data`. The `uart_receive_data()` function waits in a blocking loop `while(flagSequence == 0)`, and setting this flag allows the user function to proceed with copying the data to the user's buffer. This mechanism decouples the interrupt-driven reception from the user's blocking API, allowing the reception to occur asynchronously while providing a simple synchronization point. Third, the function re-enables the GPIO external interrupt using `HAL_NVIC_EnableIRQ(EXTI15_10_IRQn)`. Recall that the interrupt was disabled at the beginning of reception in `HAL_GPIO_EXTI_Callback()` to prevent spurious triggers during the data frame. Now that reception is complete and the interrupt flag has been cleared, it is safe to re-enable the interrupt, allowing the system to detect the start bit of the next incoming frame. Fourth and finally, the function

transitions *bufferUart.state* to EndRX using *bufferUart.state = EndRX*. On the next timer interrupt, when *uart_com_handle()* executes with this state, it will simply transition to IDLE, completing the reception cycle. This staged transition (EndRX → IDLE rather than directly to IDLE) provides a clean separation between the data processing phase and the idle phase, making the state machine easier to debug and potentially allowing for additional cleanup operations in the future if needed.

3.3.7 Flag Summary and Purpose

The following flags are used throughout the UART implementation to control state transitions and synchronize operations between the interrupt handlers and the main execution flow:

- flagStartBit:** This flag tracks whether the start bit transmission phase has completed during TX operations. It is initialized to 0 when *uart_send_byte()* is called, set to 1 after the first *timeUart = 0* phase in the TX state, and prevents the start bit logic from executing multiple times. Without this flag, the system would repeatedly reinitialize *bitIndex* and *flagDataTx* on every timer interrupt, preventing data transmission from progressing.
- flagDataTx:** This flag enables the data payload transmission phase. It is set to 1 immediately after the start bit completes and remains set throughout the 8-bit data transmission. It acts as a gate that allows the data transmission loop to execute, ensuring that data bits are only sent after the start bit has been properly transmitted.
- flagSendBit:** This flag operates within the *uart_send_bit()* helper function to create a three-phase transmission cycle for individual bits. It is set to 0 initially, transitions to 1 when the bit value is written to the TX pin at *timeUart = 0*, remains 1 during the *timeUart = 1* sampling phase, and resets to 0 at the next *timeUart = 0* to signal completion. This flag prevents the bit value from being written multiple times during a single bit period and ensures the bit remains stable for the receiver to sample.
- flagEndBit:** This flag signals when a bit transmission cycle within *uart_send_bit()* has finished. It is set to 1 when *flagSendBit* transitions from 1 to 0, indicating that a full bit period has elapsed. In the context of parity bit transmission, this flag tells the calling code in *uart_com_handle()* that the parity bit has been fully transmitted and the system can proceed to send the stop bit. Without this flag, the system would not know when to transition from parity bit transmission to stop bit transmission.
- flagStopBit:** This flag manages the stop bit transmission state in *uart_send_stop_bit()*. It follows the same three-phase pattern as *flagSendBit*: initialized to 0, set to 1 when the TX pin is driven HIGH at *timeUart = 0*, maintained at 1 during *timeUart = 1*, and reset to 0 at the next *timeUart = 0* while transitioning the state to EndTX. This ensures the stop bit lasts for exactly one bit period.
- flagParityBit:** During transmission, this flag stores the computed parity value (1 for even number of 1-bits, 0 for odd) returned by *uart_check_parity()*. It is used to determine what parity bit value to transmit based on the configured parity mode. During reception, this flag tracks whether the parity bit has been sampled and validated. It starts at 0, is set to 1 after the parity bit is read and validated (or if a parity error occurs), and prevents the parity checking logic from executing multiple times.
- flagRxStart:** This flag indicates that reception has just begun and the system is currently processing the start bit. It is set to 1 in *HAL_GPIO_EXTI_Callback()* when the falling edge is detected, and cleared to 0 after the start bit period completes (when *timeUart* cycles from 1 back to 0). This flag ensures that the system waits through the entire start bit period before beginning to sample data bits, maintaining proper timing alignment.
- flagSequence:** This flag serves as a synchronization mechanism between the interrupt-driven reception process and the user-space *uart_receive_data()* function. It is set to 1 by *reset_rx_intr()* when a complete, valid byte has been received, signaling to the blocking *uart_receive_data()* function that data is ready to be read from *bufferUart.data*. The user function waits in a loop *while(flagSequence == 0)* until this flag is set, then immediately clears it back to 0 after copying the data. This prevents the user from reading incomplete or corrupted data and ensures proper sequencing of multi-byte receptions. Critically, this flag is NOT

set during error conditions, which causes the user function to continue waiting or timeout rather than processing invalid data.

4. Problems and solutions

During the software development process, personally, I would be lying if I said I did not encounter problems. On the contrary, I can say that throughout the development I faced many issues, most of which were resolved relatively quickly with the help of the debugger. The debugger proved to be an essential tool, helping me develop the code efficiently and allowing me to run the firmware in real time in order to detect problems. I can say that the most interesting issue I encountered was related to continuous transmission. If I sent a single character from the microcontroller to the PC, or multiple characters manually, each followed by pressing Enter, everything worked correctly. However, if I typed multiple characters in a row from the keyboard, for example “aaaa”, the first character was received correctly, but the following ones were not. This error could not be detected using either the debugger or the oscilloscope. I had to logically analyze the implementation to identify the source of the problem. I concluded that there was most likely an incorrect interpretation between the stop bit of the first message and the start bit of the next one. Initially, I analyzed how continuous transmission should work, meaning that immediately after the stop bit finishes transmitting, the falling edge of the next start bit should follow. In my code, the mistake was that after the stop bit I waited for an entire bit period before attempting to receive the next bit, the same issue being present for continuous transmission as well. I modified the implementation according to the protocol specifications for successive transmissions, and the problem was resolved. As the code grew larger and the FSM became increasingly difficult to manage, I decided to create static inline functions to help with the implementation, such as functions for transmitting the stop bit and for resetting the reception parameters, since there were many cases in the flow diagram that required resetting these parameters.

5. Conclusions

In conclusion, I managed to achieve the final objectives, namely to create a UART protocol implemented purely from a software point of view, without relying on any built-in driver, developing it entirely on my own. I followed the data transmission logic of the protocol and successfully managed to transmit and receive data using the functions I implemented. I consider this project to have been a personal challenge, aimed at testing my knowledge of low-level programming, system design, and the practical application of concepts related to data communications and microcontrollers. This project is particularly important because it demonstrates that a pure software implementation of protocols that normally require hardware support is possible, without the need for a dedicated hardware driver or an external module. In this way, it becomes possible to emulate virtually any protocol, from synchronous to asynchronous ones, showcasing a flexible and practical implementation approach that can be used on microcontrollers or low-power systems that do not have a hardware UART stack integrated.

To demonstrate the functionality of the system, a test was performed. In the `main.c` file, the `uart.h` header was included and the created functions were called for configuration, transmission, and reception. The protocol was configured in software, in C, inside the `int main()` function, using ODD parity. A message with the content “pui de piropopircanita” was transmitted, after which the system waited to receive a message of up to 6 characters, with a timeout of 10 seconds. Once the message was received, it was sent back to the transmitter, repeating the same process once more, using the `uart_receive_data` and `uart_send_data` functions.

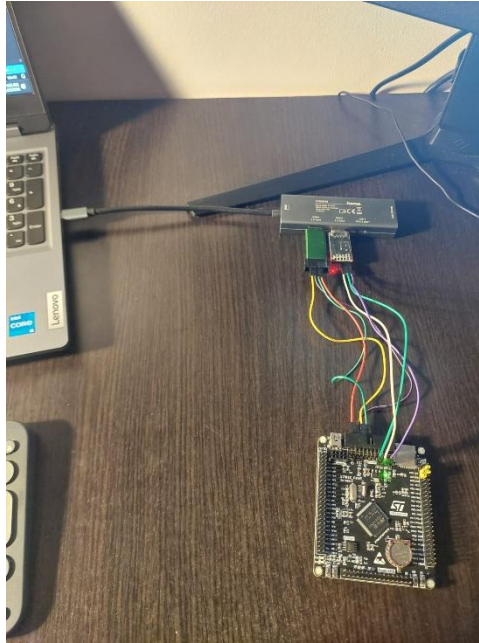


Figure16 Test Set-Up

```

93
94     uart_init(1);
95     HAL_TIM_Base_Start_IT(&htim2);
96
97     //uint8_t data = 'a';
98     //uart_send_byte(data);
99
100    uart_send_data("Pui de piropopircanita", 22);
101    char buffer[255];
102    uart_receive_data(buffer,6,10);
103    uart_send_data(buffer,6);
104    uart_receive_data(buffer,6,10);
105    uart_send_data(buffer,6);
106    /* USER CODE END 2 */
107
108    /* Infinite loop */
109    /* USER CODE BEGIN WHILE */
110    while (1)
111    {
112        /* USER CODE END WHILE */
113
114        /* USER CODE BEGIN 3 */
115    }

```

Figure17 Function used for the test in main.c

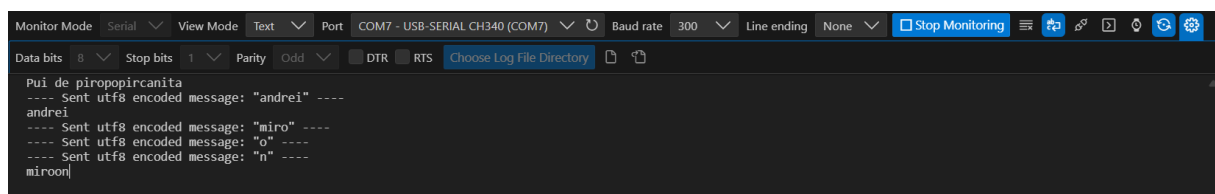


Figure18 Messages received and sent

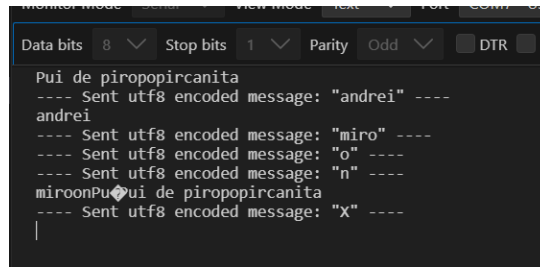


Figure19 Transmission from the PC

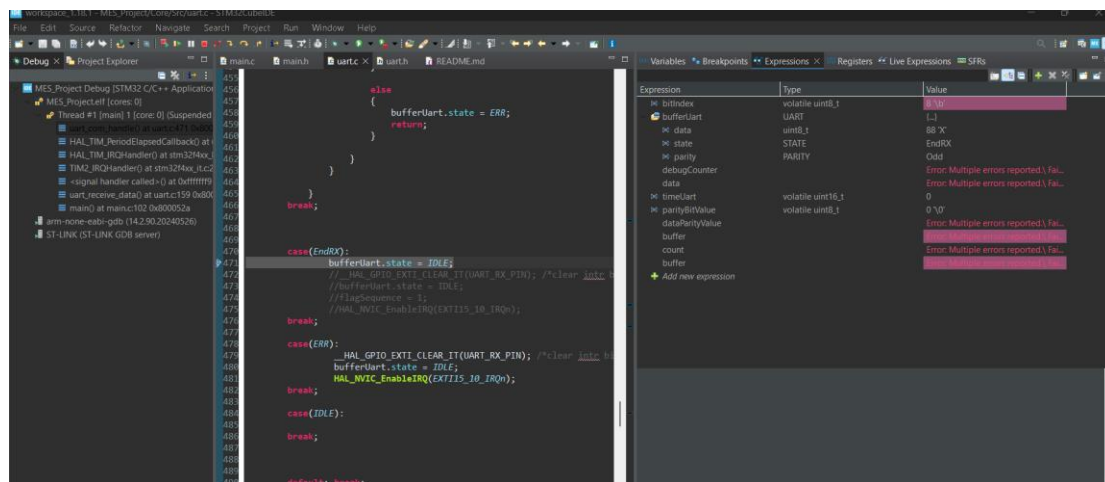


Figure20 Real Time Reception via Serial Debugger

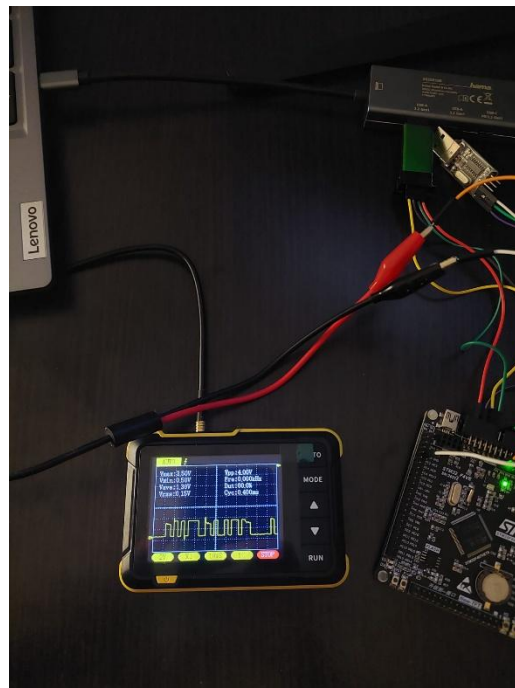


Figure21 Oscilloscope Signal View

6. Source code

Github repo: https://github.com/Myronul/MES_Project.git

uart.h

```
/*
 * uart.h
 *
 * Created on: Oct 20, 2025
 * Author: mandr
 */

#ifndef INC_UART_H_
#define INC_UART_H_

#include "main.h"
#include <stdbool.h>

void uart_init(uint8_t parity);
void uart_send_data(char* restrict data, size_t len);
void uart_receive_data(char* buffer, uint16_t len, uint16_t timeout);

#endif /* INC_UART_H_ */
```

uart.c

```
/*
 * uart.c
 *
 * Created on: Oct 20, 2025
 * Author: mandr
 */

#include "uart.h"

extern TIM_HandleTypeDef htim2;

typedef enum STATE
{
    IDLE,
    TX,
    EndTX,
    RX,
    EndRX,
    ERR
} STATE;

typedef enum PARITY
{
```

```

        None,
        Odd,
        Even
    }PARITY;

typedef struct UART
{
    uint8_t data;

    STATE state;
    PARITY parity;
}UART;

static UART bufferUart = {0,0,0}; /*init uart buffer*/
static volatile uint16_t timeUart = 0;

static volatile uint16_t timerRx = 0; /*timer for receiving data*/
static volatile uint8_t flagSequence = 0; /*flag to receive next character*/

/*flags for TX data*/
static volatile uint8_t flagStartBit = 0;
static volatile uint8_t flagStopBit = 0;
static volatile uint8_t flagDataTx = 0;
static volatile uint8_t flagSendBit = 0;
static volatile uint8_t flagEndBit = 0; /*check end send bit function*/
static volatile uint8_t flagParityBit = 0;

static volatile uint8_t parityBitValue = 0;
static volatile uint8_t bitIndex = 0;

/*flags for RX data*/
static volatile uint8_t flagRxStart = 0;

/*Functions prototypes*/
void uart_init(uint8_t parity);
void uart_send_data(char* restrict data, size_t len);
void uart_receive_data(char* buffer, uint16_t len, uint16_t timeout);
static inline void uart_send_byte(uint8_t data);
static inline uint8_t uart_check_parity(uint8_t data);
static inline void uart_com_handle(void);
static inline void reset_rx_intr(void);

void uart_init(uint8_t parity)
{
    /*
     * init of the uart cusotm protocol
     * Input: parity type: 0 none, 1 odd and 2 even
     * Output: void
     */

    bufferUart.parity = parity;

```

```

        bufferUart.state = IDLE;

        HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_SET);
    }

static inline void uart_send_byte(uint8_t data)
{
    /*
     * Function to change the state in sending data. The effective sending
     * will have place in the interrupt function call
     * Input: data of unsigned 8 bit
     * Output: void
     */

    bufferUart.data = data;

    flagStartBit = 0;
    flagDataTx = 0;
    flagSendBit = 0;
    flagEndBit = 0;

    if(bufferUart.parity != None)
    {
        flagParityBit = uart_check_parity(bufferUart.data);
    }

    TIM2->CNT = 0; /*reset counter register*/
    timeUart = 0;

    HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_RESET);
    bufferUart.state = TX;
}

void uart_send_data(char* restrict data, size_t len)
{
    /*
     * Function to send a large number of data.
     * Input: Pointer to the data of 8 bit data type
     *          Length of the data packet
     * Output: void
     */

    while(len != 0)
    {
        while(bufferUart.state == TX || bufferUart.state == EndTX); /*wait for
idle state*/
        uart_send_byte(*data);
        data++;
        len--;
    }
}

```

```

void uart_receive_data(char* buffer, uint16_t len, uint16_t timeout)
{
    /*
     * Function that will wait a certain timeout to receive data,
     * via UART.
     * Input: Pointer to a 8 bit data
     *          Length of the data in bytes
     *          timeout in seconds
     */

    timerRx = timeout*600; /*1 seconds*/

    while(len!=0 && timerRx!=0)
    {
        while(flagSequence == 0);
        flagSequence = 0;
        *buffer = bufferUart.data;
        buffer++;
        len--;
    }
}

```

```

static inline uint8_t uart_check_parity(uint8_t data)
{
    /*
     * Function that will return the parity of the
     * data frame response. It will return true for
     * even parity and false for odd parity in O(1) time
     * Input: 8 bit data value
     * Output: uint8_t 0 or 1
     */

    data = data - ((data>>1)&0x55);
    data = (data&0x33) + (data>>2&0x33);
    data = (data + (data>>4)) & 0x0F;

    if(data % 2 == 0)
    {
        return true; /*1*/
    }

    else
    {
        return false; /*0*/
    }
}

```

```

static void uart_send_stop_bit(void)
{
    /*
     * Static function to send the last bit.

```

```

    * timeUart must be equal to 0, at the
    * beginning of the new bit. The function
    * will change the state of the bufferUart
    * variable.
    * Input: void
    * Output: void
    */

    if(timeUart == 0 && flagStopBit == 0)
    {
        flagStopBit = 1;
        HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, GPIO_PIN_SET);
        return;
    }

    if(timeUart==1 && flagStopBit == 1)
    {
        return; /*make sure to end stop bit*/
    }

    if(timeUart == 0 && flagStopBit == 1)
    {
        flagStopBit = 0;
        bufferUart.state = EndTX;
    }

}

static inline void uart_send_bit(uint8_t bit)
{
    /*
    * Static function to send a bit of
    * 1 or 0. TimeUart must be equal to
    * 0, at the start of a new bit sample
    * Input: bit value 1 or 0
    * Output: void
    */

    if(timeUart == 0 && flagSendBit == 0)
    {
        flagSendBit = 1;
        HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN, bit);
        return;
    }

    if(timeUart==1 && flagSendBit == 1)
    {
        return; /*make sure to end stop bit*/
    }

    if(timeUart == 0 && flagSendBit == 1)
    {
        flagSendBit = 0;
        flagEndBit = 1;
    }
}

```

```

}

static inline void reset_rx_intr(void)
{
    __HAL_GPIO_EXTI_CLEAR_IT(UART_RX_PIN);
    flagSequence = 1;
    HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
    bufferUart.state = EndRX;
}

static inline void uart_com_handle(void)
{
    switch(bufferUart.state)
    {
        case(TX):

            if(timeUart == 0 && flagStartBit == 0)
            {
                /*end start bit tx*/

                flagStartBit = 1;
                flagDataTx = 1;
                bitIndex = 0;

            }

            if(flagDataTx == 1)
            {
                /*start data payload transmission*/

                if(bitIndex < 8 && timeUart == 0)
                {
                    if((bufferUart.data & (1<<bitIndex)) != 0) /*1*/
                    {
                        HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN,
GPIO_PIN_SET);
                    }

                    else /*0*/
                    {
                        HAL_GPIO_WritePin(UART_TX_PORT, UART_TX_PIN,
GPIO_PIN_RESET);
                    }

                    bitIndex++;

                    return; /*the return will make the 8th bit
                                last all cycle until timeUart is 0*/
                }

                /*parity bit tx*/

                if(bitIndex == 8 && timeUart == 0) /*first iteration is
0*/
                {
                    if(bufferUart.parity == None)

```

```

        {
            uart_send_stop_bit();
            return;
        }

        if(flagEndBit == 0)
        {
            if(bufferUart.parity == Even &&
flagParityBit==true)
            {
                uart_send_bit(0);
            }

            if(bufferUart.parity == Even &&
flagParityBit==false)
            {
                uart_send_bit(1);
            }

            if(bufferUart.parity == Odd &&
flagParityBit==true)
            {
                uart_send_bit(1);
            }

            if(bufferUart.parity == Odd &&
flagParityBit==false)
            {
                uart_send_bit(0);
            }
        }

        if(flagEndBit == 1)
        {
            uart_send_stop_bit(); /*iteration of
timerUart equal 0*/
        }
    }

    break;

    case (EndTX) :
        bufferUart.state = IDLE;
        break;

    case (RX) :

        if(flagRxStart==1 && timeUart==1)
        {
            /*start bit, wait...*/
            return;
        }

```



```

else
{
    flagRxStart = 0;
}

if(bitIndex < 8 && timeUart == 1)
{
    /*receive the message at time 1*/

    if(HAL_GPIO_ReadPin(UART_RX_PORT, UART_RX_PIN) == 1)
    {
        /*HIGH -> 1*/
        bufferUart.data = bufferUart.data | (1<<bitIndex);
    }

    else
    {
        /*LOW -> 0*/
        bufferUart.data = bufferUart.data & ~(1<<bitIndex);
    }

    bitIndex++;

    return; /*the return will make the 8th bit last*/
    /*all cycle until 0 uart time*/
}

if(bitIndex == 8 && timeUart == 1)
{
    /*end rx transmission*/

    if(bufferUart.parity == None)
    {
        /*stop bit*/
        if(HAL_GPIO_ReadPin(UART_RX_PORT, UART_RX_PIN) ==
1)
        {
            //bufferUart.state = EndRX;
            reset_rx_intr();
        }

        else
        {
            bufferUart.state = ERR;
            return;
        }
    }

    else
    {
        if(flagParityBit == 0) /*parity bit sampling*/
        {
            /*take parity bit value*/
            parityBitValue =
HAL_GPIO_ReadPin(UART_RX_PORT, UART_RX_PIN);

```

```

        if(((uart_check_parity(bufferUart.data) +
parityBitValue) % 2 == 1) && (bufferUart.parity == Even))
        {
            /*correct bit sampling*/
            flagParityBit = 1;
            return;
        }

        if(((uart_check_parity(bufferUart.data) +
parityBitValue) % 2 == 0) && (bufferUart.parity == Odd))
        {
            /*correct bit sampling*/
            flagParityBit = 1;
            return;
        }

        /*error if not*/
        bufferUart.state = ERR;
        flagParityBit = 1;
        return;
    }

    if(flagParityBit == 1)
    {
        /*stop bit*/

        if(HAL_GPIO_ReadPin(UART_RX_PORT,
UART_RX_PIN) == 1)
        {
            reset_rx_intr();
        }

        else
        {
            bufferUart.state = ERR;
            return;
        }
    }
}

    }
break;

case (EndRX) :
    bufferUart.state = IDLE;
    //__HAL_GPIO_EXTI_CLEAR_IT(UART_RX_PIN); /*clear intr bit flag*/
    //bufferUart.state = IDLE;
    //flagSequence = 1;
    //HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
break;

case (ERR) :
    __HAL_GPIO_EXTI_CLEAR_IT(UART_RX_PIN); /*clear intr bit flag*/
    bufferUart.state = IDLE;
    HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
break;

```

```

        case (IDLE) :

            break;

        default: break;

    }

}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    /*
     * Weak function ISR for the timer callback
     */

    if(htim->Instance == TIM2)
    {
        timeUart = (timeUart + 1) % 2;
        uart_com_handle();
        //HAL_GPIO_TogglePin(UART_TX_PORT, UART_TX_PIN);
        if(timerRx != 0)
        {
            timerRx--;
        }
    }
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    /*
     * Weak function ISR for the interrupt RX callback
     */

    if(GPIO_Pin == UART_RX_PIN)
    {
        /*start reception data*/
        HAL_NVIC_DisableIRQ(EXTI15_10_IRQn);

        flagParityBit = 0; /*reset flag for rx stop bit*/
        flagRxStart = 1;
        timeUart = 0;
        bitIndex = 0;
        bufferUart.data = 0; /*init data*/
        TIM2->CNT = 0;

        bufferUart.state = RX;
    }
}

```

Obs:

- The project presentations will be on 22.01.2026 (15:00 – 20:00) for ACES and on 23.01.2026 for AM (15:00 – 20:00).
- You can add chapters or subsections to the documentation, if needed.
- You must prepare a short presentation containing the essential information from your project (no more than 10 minutes presentation + questions).
- You must prepare a functional prototype of your project. For special cases, a video with the working prototype can be accepted.
- You must create an archive with your word document (also a pdf version), power point presentation and the source code and upload it on Moodle until 19.01.2026.
- The maximum grade for the project will be 50 points (half of the final grade).